# Procesamiento Masivo de Datos

Tarea 2: Redis + HBase — del stream a la historia

ELE051-B / EIN102B · USM 2026-2

**Profesor:** Diego Ramírez
**Asignación:** por definir (se informa por Aula)
**Modalidad:** grupos de máximo 2 a 3 personas
**Integrantes:** _____

**Paralelo:** 701
**Entrega:** domingo 8 de noviembre de 2026, 23:59
**Vía:** Aula
_____

## Instrucciones

- Lea el enunciado completo antes de empezar. La Parte B (el pipeline) condiciona todo lo que sigue.
- Se evalúa el repositorio **tal como queda entregado**: se corrige en una máquina limpia, sin editar rutas ni correr pasos no escritos.
- Justifique sus decisiones técnicas. Un resultado sin interpretación, o una medición sin su tiempo, no suma puntaje.
- Los datos **no** se versionan: se generan con la semilla. El repositorio debe traer un `.gitignore`.
- Se permite reutilizar sin restricción el código de los Laboratorios 06 y 07. Lo que se evalúa es lo que ustedes construyen encima.

**Puntaje Total:** 100 puntos (20 Infraestructura + 25 Pipeline + 30 Preguntas + 15 Fallas + 10 Informe)
**Ponderación:** 20 % de la nota final

## 1. La idea

En los Laboratorios 06 y 07 vieron los dos extremos de un mismo pipeline: **Redis** guarda el presente en memoria (la última lectura, el ranking de ahora, el stream de la última hora) y **HBase** guarda la historia en disco (años de lecturas, repartidos en regiones). Hasta ahora los usaron por separado y con el código a medio escribir. La tarea es **construir el pipeline completo, solos, y decidir qué va en cada lado**: los eventos entran a un stream de Redis; un grupo de consumidores los baja a HBase sin perder ni duplicar nada; otro grupo mantiene el tablero "de ahora" en Redis; y un conjunto de preguntas se responde desde el motor que corresponde — **midiendo** qué cuesta equivocarse. Es, en chico y con datos dados, la parte de streaming + NoSQL del proyecto semestral.

La pregunta que ordena todo: **¿qué pregunta le toca a cada motor, y cuánto cuesta ponerla en el equivocado?**

## 2. Los datos (comunes, generados con semilla fija)

Los sensores del Laboratorio 06: 16 estaciones del Gran Concepción, tres tipos de sensor, ∼2 eventos por segundo. El generador es el mismo (`Laboratorios/06/generador.py`; pueden copiarlo al repositorio: es código, no datos), pero con **parámetros propios de la tarea**, para que ninguna respuesta se pueda copiar de los notebooks:

| Parámetro | Valor |
|---|---|
| Llamada | `eventos(t0, semilla=2026)` — ojo: la semilla por defecto del generador es 2509, **no** la de la tarea |
| `t0` | `2026-11-02T00:00:00-03:00` |
| **Histórico** | los primeros **864.000** eventos (5 días × 172.800) |
| **En vivo** | los **7.200** eventos siguientes (una hora) |
| Ritmo del en vivo | 20× el real: un evento cada 25 ms (la hora entera en ∼3 min) |
| Comuna de cada estación | la del prefijo del `estacion_id`: `CC` Concepción · `TAL` Talcahuano · `HUA` Hualpén · `SPP` San Pedro de la Paz · `CHI` Chiguayante · `PEN` Penco · `COR` Coronel |

**Números de control.** Si su carga no da esto, algo está mal (y saber explicar por qué no da también cuenta):

| | Histórico | En vivo | Total |
|---|---:|---:|---:|
| Eventos generados | 864.000 | 7.200 | 871.200 |
| Repetidos tal cual (los cuatro campos iguales a otro evento) | 17.090 | 153 | 17.243 |
| Claves distintas (`estacion_id` + `timestamp`) | 846.906 | 7.047 | **853.953** |

La diferencia entre 871.200 y 853.953 son los 17.243 repetidos más **4 pares de eventos con la misma estación y el mismo milisegundo pero distinto valor** (dos lecturas distintas que chocan en la clave). Con una clave así, HBase se queda con el último valor escrito: no es un error de su pipeline y la pauta lo tolera.

Los eventos traen, además, **lecturas tardías** (∼3 %: el `timestamp` de medición es hasta dos minutos anterior a cuando llegan). Son parte del problema: la clave de HBase y el "último valor" de Redis tienen que decidir con cuál de los dos relojes trabajar (Lab 06, `CONTENTS.md` §5.5).

**No usen el VPS del curso.** El servidor compartido puede no estar disponible cuando se corrija. Todo corre en la máquina de ustedes, con Redis y HBase locales.

## 3. Qué hay que entregar

Un **repositorio git público** y un **informe breve** en PDF. Por Aula se entrega el **link del repositorio** y el PDF del informe. El repositorio tiene que estar público y accesible a la hora de entrega: si al corregir el link no abre, la tarea no se corrige.

El repositorio debe contener:

1. `docker-compose.yml` (+ el `Dockerfile` de HBase del Lab 07) que levante **Redis y HBase**.
2. **El código del pipeline**: la carga histórica, el productor, los dos consumidores, la verificación y las consultas. Un solo comando (`./run.sh`, `make` o equivalente, documentado) corre todo de punta a punta sobre una máquina limpia, con los contenedores arriba.

3. **Las preguntas** (notebook o scripts), con resultados y **tiempos medidos**.
4. `README.md` con: cómo reproducir desde cero y cuánto tarda, las decisiones tomadas (row keys, regiones, `MAXLEN`, persistencia de Redis) y qué **no** funcionó.

El informe (máximo **5 páginas**, PDF): las respuestas a las preguntas, los experimentos de la Parte D y una tabla "qué va en cada motor".

## 4. Parte A — Infraestructura reproducible (20 puntos)

- **Dos servicios** en compose: `redis` (7) y `hbase` (el del Lab 07, con su volumen con nombre). Healthchecks, de modo que `docker compose up -d --wait` espere a que ambos respondan.
- **Persistencia de Redis: una decisión explícita**, escrita en el compose (RDB, AOF o ninguna) y justificada en el README. Se pone a prueba en la Parte D.
- Un solo script deja todo cargado y verificado. Se prueba **tal cual está en el repositorio**: si hay que editar una ruta o correr cosas en un orden no escrito, no es reproducible.
- Python en ambiente virtual, con `requirements.txt` (`redis`, `happybase`, `pandas`, lo que usen).
- **Sin datos en el repositorio**: los datos se generan con la semilla. `.gitignore` incluido (y nada de claves ni `.env` con secretos).

## 5. Parte B — El pipeline (25 puntos)

### B.1 Carga histórica (7 puntos)

Los 864.000 eventos del histórico van **directo a HBase** (sin pasar por Redis: es la historia que ya existe). Con `batch`, no `put` por `put`; reporten el tiempo y las filas por segundo.

- Mínimo **dos tablas**, pre-divididas en regiones (`SPLITS`), con los cortes justificados:
  - `lecturas`: para "lecturas de una estación en un rango de tiempo".
  - `lecturas_tipo`: para "lecturas de un tipo de sensor en un rango de tiempo" (se usa en P4).
- La **row key** de cada tabla, en el README: qué pregunta hace rápida, cuál hace lenta, y por qué es **idempotente** (la clave sale de la estación y de la *medición*, nunca de la llegada: así un evento repetido o tardío cae en su lugar).

### B.2 El camino en vivo (10 puntos)

- `productor`: lee la hora en vivo y la escribe en el stream `sensores:t2` con `XADD`, a 20×. Con un `MAXLEN ~` que justifiquen: ¿cuánto atraso (lag) del consumidor tolera? ¿Qué pasa con una entrada recortada que nadie leyó?
- **Dos grupos de consumidores independientes** sobre el mismo stream (esa es la gracia de los consumer groups: cada grupo ve *todo* el stream):

- `ingesta`: lee en lotes y escribe en HBase (las dos tablas). El `XACK` **va después del** `put`, nunca antes.
- `tablero`: mantiene en Redis el estado "de ahora". Al terminar la hora en vivo tiene que existir (con estos nombres, para que se pueda revisar con `redis-cli`):
  - `ultimo:<estacion>` — un hash con `valor`, `ts` (medición, en ms) y `tipo`.
  - `ranking:flujo` — un sorted set `estacion` → último valor, solo estaciones de flujo vehicular.
  - `ventana:<estacion>` — un sorted set con `score = ts de medición (ms)`, solo flujo vehicular, recortado a los **últimos 10 minutos de medición**.
  - `tipo:<tipo>` — un set con las estaciones de cada tipo de sensor (el catálogo).

- **"Último" significa último por medición.** Una lectura tardía no puede hacer retroceder el tablero. Documenten cómo lo logran.

### B.3 Verificación (8 puntos)

Un `verificar.py` (o equivalente) que, al terminar todo, imprima: eventos producidos · entradas procesadas y `XPENDING` de cada grupo (debe ser 0) · filas en cada tabla de HBase · y las compare con los números de control. Si algo no calza, que lo diga y lo explique.

## 6. Parte C — Preguntas (30 puntos)

Seis preguntas, 5 puntos cada una. Para **cada una**: la consulta o el código, el resultado, el **tiempo medido** (la mediana de 5 corridas, descartando la primera: calienta cachés) y **una frase de interpretación**.

1. **"Ahora" — Redis contra HBase.** Al terminar la hora en vivo: el último valor de cada una de las 16 estaciones, y las 5 estaciones de flujo vehicular con más flujo. Respóndanla **desde Redis y desde HBase**, comparen los tiempos y digan cuál usaría un dashboard, y por qué. (En HBase, "lo último" no sale gratis con la clave `estacion#timestamp`: hay que decidir cómo.) **Una estación** cambia de respuesta según se tome el último *por medición* o el último *por llegada*: identifíquenla y muestren las dos lecturas.

2. **Ventana de 10 minutos — Redis.** El promedio de flujo de los últimos 10 minutos de medición (anclados a la medición más reciente del stream) para cada estación de flujo vehicular, **leyendo los** `ventana:*` — no con los datos en memoria de Python ni desde HBase. ¿Por qué un sorted set con el timestamp como puntaje? ¿Qué pasaría con la memoria si no recortaran la ventana? (`MEMORY USAGE`).

3. **Un rango — HBase.** Las lecturas de **CC-001 el 04-nov entre las 08:00 y las 10:00** (rango semiabierto: 08:00 incluido, 10:00 excluido): cuántas son y su promedio por cuarto de hora. Con un `scan` por rango y, para comparar, con un recorrido de la tabla entera más un filtro. Los dos tiempos, cuántas veces más lento es el segundo, y qué esperarían con cinco años de datos en vez de cinco días.

4. **Dos patrones de acceso — Redis catálogo + HBase.** Para cada estación de **calidad del aire**: promedio y máximo de PM2.5 entre las **07:00 y las 09:00 del 05-nov**, y la hora del máximo (si el máximo se repite, la primera ocurrencia). Dos veces:

a) En `lecturas` (estación primero): un rango por estación, con la lista de estaciones sacada del catálogo de Redis (`tipo:calidad_aire`).

b) En `lecturas_tipo` (tipo primero): un solo `scan`.

Los tiempos y, además, **lo que cuesta tener la segunda tabla**: escrituras extra, espacio en disco, y qué pasa cuando llega un evento (hay que escribir dos veces: ¿y si falla la segunda?). Digan en qué escenario no valdría la pena.

5. **Hotspotting y salting — HBase.** La hora en vivo, escrita en **tres tablas** con la misma información y distinta clave: `estacion#ts` · `ts#estacion` (pre-dividida por día) · `salt#ts#estacion`, con `salt = hash(estacion) mod 4` (pre-dividida por salt). Midan **cuántas escrituras recibió cada región** (las métricas JMX de HBase, como en el Lab 07, 3.2) y armen una tabla de 3 × regiones. Después, el tiempo de responder "**todas las lecturas de todas las estaciones entre las 00:10 y las 00:20 del 07-nov**" con cada diseño (16 rangos, 1 rango, 4 rangos). ¿Cuál elegirían, y bajo qué supuestos cambiarían de idea?

6. **La propia.** Una pregunta que ustedes formulen, que necesite **Redis y HBase a la vez** (uno decide el *qué*, el otro el *cuándo*, o al revés) y que no sea una variante de las anteriores. Justifiquen el reparto. (Una idea de partida, para descartarla si quieren: "estaciones que ahora promedian más que su promedio histórico a esa misma hora".)

## 7. Parte D — Cuando algo se cae (15 puntos)

### D.1 El consumidor muere a mitad de lote (8 puntos)

Hagan que `ingesta` **muera a propósito** (`os._exit(1)` o `kill -9`) después del `put` de un lote y **antes** del `XACK`. Muestren:

1. Qué quedó en `XPENDING`: cuántas entradas, a nombre de qué consumidor.
2. Otro consumidor que las recupera (`XAUTOCLAIM` o `XREADGROUP` con id `0`).
3. Que al final HBase tiene **exactamente** lo que dicen los números de control: ni faltan ni sobran filas.
4. Por qué reprocesar el lote **no duplica filas** — y qué quedó guardado en la celda (versiones).

### D.2 Redis se reinicia (7 puntos)

Con el pipeline a medio camino (por ejemplo, con unos 3.000 eventos ya producidos), `docker compose restart redis`, con **dos configuraciones**: la que ustedes eligieron en la Parte A y su contraria (sin persistencia ↔ AOF con volumen). Para cada una, qué sobrevivió: el stream, los grupos de consumidores y sus pendientes, los `ultimo:*`, las ventanas.

Y la pregunta de diseño: ¿cuál es la **ventana de pérdida** — los eventos que están en Redis y todavía no en HBase? ¿Qué parte de la historia está a salvo en cada momento? ¿Qué pérdida máxima aceptarían en un sistema real, y qué configuración (`appendfsync`) eligen por eso?

## 8. Parte E — Informe (10 puntos)

Máximo 5 páginas. Se evalúa:

- **Claridad:** cada respuesta se entiende sin abrir el código.
- **Tabla "qué va en cada motor":** para cada dato de este problema (la última lectura, la ventana de 10 minutos, la historia por estación, el catálogo de estaciones, el stream, la historia por tipo...): en qué motor, con qué estructura o clave, y por qué ahí y no en el otro.
- **Tabla de tiempos** (P1, P3, P4, P5), con el computador en que se midió.
- **Honestidad con la escala:** HBase *standalone* en un PC no es un cluster. Las conclusiones dicen hasta dónde se extrapolan las mediciones y desde dónde ya no.
- **Qué no funcionó** y qué harían distinto.

## 9. Rúbrica (sobre 100)

| Criterio | Pts | Qué se mira |
|---|---|---|
| A. Infraestructura reproducible | 20 | Levanta con `docker compose up -d --wait` + un script, en una máquina limpia, sin editar nada. Persistencia de Redis decidida y justificada. README. |
| B. El pipeline | 25 | Carga con `batch`; tablas pre-divididas y row keys justificadas e idempotentes; dos grupos independientes; `XACK` después del `put`; "último por medición"; los números de control calzan. |
| C. Preguntas | 30 | Las 6: correctas, desde el motor adecuado, con el tiempo medido (mediana de 5) y la interpretación. La propia necesita de verdad a los dos motores. |
| D. Cuando algo se cae | 15 | El experimento realmente mata al consumidor entre `put` y `XACK`; se recupera sin pérdida ni duplicados; D.2 compara las dos configuraciones con evidencia y razona la ventana de pérdida. |
| E. Informe | 10 | Se entiende, la tabla "qué va en cada motor" está bien razonada, los tiempos tienen su hardware, se dice hasta dónde llega la escala. |

**Descuentos fijos:**

- No levanta con compose (o requiere pasos no escritos) → **máximo 60** en total.
- Datos versionados en el repositorio (parquets, dumps, volúmenes) o claves/secretos → −10.
- Entrega fuera de plazo → −10 **por día**, máximo 3 días.

## 10. Consejos y riesgos

- **Construyan hoy la imagen de HBase** (`docker compose build`, ∼340 MB, una vez por máquina): es lo único que depende de la red. Necesita ∼4 GB de RAM asignados a Docker y ∼3 GB de disco.

- **La carga histórica tarda** (en el Lab 07, ∼1,2 millones de lecturas en ∼3 min en un Mac reciente; 2–3× en un PC más lento). Durante el desarrollo no la repitan en cada prueba — el volumen conserva las tablas —, pero el script final tiene que poder correr desde cero.
- `batch`**, no** `put` **por fila.** Y para contar filas, `FirstKeyOnlyFilter() AND KeyOnlyFilter()` (Lab 07, 2.2).
- **Las row keys se comparan como bytes.** Los timestamps ISO con ceros y con el mismo offset (`-03:00`) ordenan bien; los números hay que rellenarlos a ancho fijo (Lab 07, 3.3). No mezclen UTC con hora de Chile.
- `XACK` **después del** `put`**.** Si lo invierten, el experimento D.1 les va a mostrar por qué se pierden eventos.
- **JMX publica cada ∼10 s** (Lab 07, 3.2): esperen antes de leer los contadores de región.
- **Midan bien:** mediana de 5 corridas, la primera aparte. Lo que se compara son órdenes de magnitud, no milisegundos.
- **Los repetidos y las tardías no son ruido:** son la razón de ser de la idempotencia. Un conteo "demasiado bajo" suele ser una clave bien diseñada.
- **Preguntas:** en clase, o por el canal del curso. Las respuestas que sirvan a todos se publican para todos.